

12

Animation Systems

The majority of modern 3D games revolve around *characters*—often human or humanoid, sometimes animal or alien. Characters are unique because they need to move in a fluid, organic way. This poses a host of new technical challenges, over and above what is required to simulate and animate rigid objects like vehicles, projectiles, soccer balls and Tetris pieces. The task of imbuing characters with natural-looking motion is handled by an engine component known as the *character animation system*.

As we'll see, an animation system gives game designers a powerful suite of tools that can be applied to non-characters as well as characters. Any game object that is not 100% rigid can take advantage of the animation system. So whenever you see a vehicle with moving parts, a piece of articulated machinery, trees waving gently in the breeze or even an exploding building in a game, chances are good that the object makes at least partial use of the game engine's animation system.

12.1 Types of Character Animation

Character animation technology has come a long way since *Donkey Kong*. At first, games employed very simple techniques to provide the illusion of lifelike movement. As game hardware improved, more-advanced techniques became

feasible in real time. Today, game designers have a host of powerful animation methods at their disposal. In this section, we'll take a brief look at the evolution of character animation and outline the three most-common techniques used in modern game engines.

12.1.1 Cel Animation

The precursor to all game animation techniques is known as *traditional animation*, or *hand-drawn animation*. This is the technique used in the earliest animated cartoons. The illusion of motion is produced by displaying a sequence of still pictures known as *frames* in rapid succession. Real-time 3D rendering can be thought of as an electronic form of traditional animation, in that a sequence of still full-screen images is presented to the viewer over and over to produce the illusion of motion.

Cel animation is a specific type of traditional animation. A *cel* is a transparent sheet of plastic on which images can be painted or drawn. An animated sequence of cels can be placed on top of a fixed background painting or drawing to produce the illusion of motion without having to redraw the static background over and over.

The electronic equivalent to cel animation is a technology known as *sprite animation*. A *sprite* is a small bitmap that can be overlaid on top of a full-screen background image without disrupting it, often drawn with the aid of specialized graphics hardware. Hence, a sprite is to 2D game animation what a cel was to traditional animation. This technique was a staple during the 2D game era. Figure 12.1 shows the famous sequence of sprite bitmaps that were used to produce the illusion of a running humanoid character in almost every Mattel Intellivision game ever made. The sequence of frames was designed so that it animates smoothly even when it is repeated indefinitely—this is known as a *looping animation*. This particular animation would be called a *run cycle* in modern parlance, because it makes the character appear to be running. Characters typically have a number of looping animation cycles, including various idle cycles, a walk cycle and a run cycle.



Figure 12.1. The sequence of sprite bitmaps used in most Intellivision games.

12.1.2 Rigid Hierarchical Animation

Early 3D games like *Doom* continued to make use of a sprite-like animation system: Its monsters were nothing more than camera-facing quads, each of which displayed a sequence of texture bitmaps (known as an *animated texture*) to produce the illusion of motion. And this technique is still used today for low-resolution and/or distant objects—for example crowds in a stadium, or hordes of soldiers fighting a distant battle in the background. But for high-quality foreground characters, 3D graphics brought with it the need for improved character animation methods.

The earliest approach to 3D character animation is a technique known as *rigid hierarchical animation*. In this approach, a character is modeled as a collection of rigid pieces. A typical breakdown for a humanoid character might be pelvis, torso, upper arms, lower arms, upper legs, lower legs, hands, feet and head. The rigid pieces are constrained to one another in a hierarchical fashion, analogous to the manner in which a mammal's bones are connected at the joints. This allows the character to move naturally. For example, when the upper arm is moved, the lower arm and hand will automatically follow it. A typical hierarchy has the pelvis at the root, with the torso and upper legs as its immediate children and so on as shown below:

```
Pelvis
  Torso
    UpperRightArm
      LowerRightArm
        RightHand
    UpperLeftArm
      UpperLeftArm
        LeftHand
  Head
  UpperRightLeg
    LowerRightLeg
      RightFoot
  UpperLeftLeg
    UpperLeftLeg
      LeftFoot
```

The big problem with the rigid hierarchy technique is that the behavior of the character's body is often not very pleasing due to "cracking" at the joints. This is illustrated in Figure 12.2. Rigid hierarchical animation works well for robots and machinery that really are constructed of rigid parts, but it breaks down under scrutiny when applied to "fleshy" characters.

12.1.3 Per-Vertex Animation and Morph Targets

Rigid hierarchical animation tends to look unnatural because it is rigid. What we really want is a way to move individual vertices so that triangles can stretch to produce more natural-looking motion.

One way to achieve this is to apply a brute-force technique known as *per-vertex animation*. In this approach, the vertices of the mesh are animated by an artist, and motion data is exported, which tells the game engine how to move each vertex at runtime. This technique can produce any mesh deformation imaginable (limited only by the tessellation of the surface). However, it is a data-intensive technique, since time-varying motion information must be stored for each vertex of the mesh. For this reason, it has little application to real-time games.

A variation on this technique known as *morph target animation* is used in some real-time games. In this approach, the vertices of a mesh are moved by an animator to create a relatively small set of fixed, extreme poses. Animations are produced by *blending* between two or more of these fixed poses at runtime. The position of each vertex is calculated using a simple linear interpolation (LERP) between the vertex's positions in each of the extreme poses.

The morph target technique is often used for facial animation, because the human face is an extremely complex piece of anatomy, driven by roughly 50 muscles. Morph target animation gives an animator full control over every vertex of a facial mesh, allowing him or her to produce both subtle and extreme movements that approximate the musculature of the face well. Figure 12.3 shows a set of facial morph targets.

As computing power continues to increase, some studios are using jointed facial rigs containing hundreds of joints as an alternative to morph targets. Other studios combine the two techniques, using jointed rigs to achieve the primary pose of the face and then applying small tweaks via morph targets.

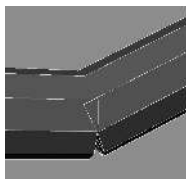


Figure 12.2. Cracking at the joints is a big problem in rigid hierarchical animation.

12.1.4 Skinned Animation

As the capabilities of game hardware improved further, an animation technology known as *skinned animation* was developed. This technique has many of the benefits of per-vertex and morph target animation—permitting the triangles of an animated mesh to deform. But it also enjoys the much more efficient performance and memory usage characteristics of rigid hierarchical animation. It is capable of producing reasonably realistic approximations to the movement of skin and clothing.

Skinned animation was first used by games like *Super Mario 64*, and it is still the most prevalent technique in use today, both by the game industry and the feature film industry. A host of famous modern game and movie characters, including the dinosaurs from *Jurrassic Park*, Solid Snake (*Metal Gear Solid 4*), Gollum (*Lord of the Rings*), Nathan Drake (*Uncharted*), Buzz Lightyear (*Toy Story*), Marcus Fenix (*Gears of War*) and Joel (*The Last of Us*) were all animated, in whole or in part, using skinned animation techniques. The remainder of this chapter will be devoted primarily to the study of skinned/skeletal animation.

In skinned animation, a *skeleton* is constructed from rigid “bones,” just as in rigid hierarchical animation. However, instead of rendering the rigid pieces on-screen, they remain hidden. A smooth continuous triangle mesh called a *skin* is bound to the joints of the skeleton; its vertices track the movements of the joints. Each vertex of the skin mesh can be weighted to multiple joints, so the skin can stretch in a natural way as the joints move.

In Figure 12.4, we see Crank the Weasel, a game character designed by Eric Browning for Midway Home Entertainment in 2001. Crank’s outer skin is composed of a mesh of triangles, just like any other 3D model. However, inside him we can see the rigid bones and joints that make his skin move.



Figure 12.3. A set of facial morph targets for the Ellie character in *The Last of Us: Remastered* (© 2014/™ SIE. Created and developed by Naughty Dog, PlayStation 4).

12.1.5 Animation Methods as Data Compression Techniques

The most flexible animation system conceivable would give the animator control over literally every infinitesimal point on an object's surface. Of course, animating like this would result in an animation that contains a potentially infinite amount of data! Animating the vertices of a triangle mesh is a simplification of this ideal—in effect, we are *compressing* the amount of information needed to describe an animation by restricting ourselves to moving only the vertices. (Animating a set of control points is the analog of vertex animation for models constructed out of higher-order patches.) Morph targets can be thought of as an additional level of compression, achieved by imposing additional constraints on the system—vertices are constrained to move only along linear paths between a fixed number of predefined vertex positions. Skeletal animation is just another way to compress vertex animation data by imposing constraints. In this case, the motions of a relatively large number of vertices are constrained to follow the motions of a relatively small number of skeletal joints.

When considering the trade-offs between various animation techniques, it can be helpful to think of them as compression methods, analogous in many respects to video compression techniques. We should generally aim to select the animation method that provides the best compression without producing unacceptable visual artifacts. Skeletal animation provides the best compression when the motion of a single joint is magnified into the motions of many vertices. A character's limbs act like rigid bodies for the most part, so they can

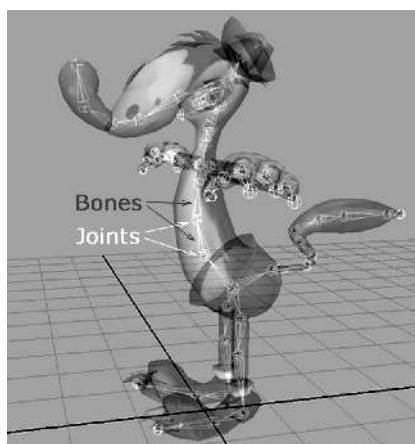


Figure 12.4. Eric Browning's Crank the Weasel character, with internal skeletal structure.

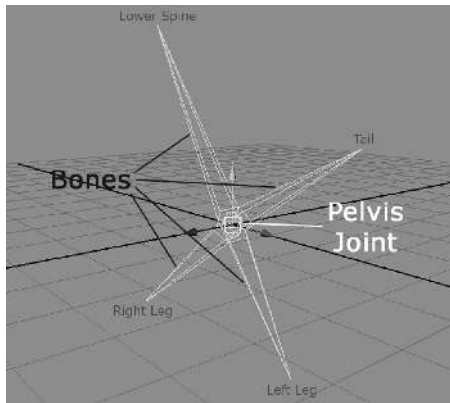


Figure 12.5. The pelvis joint of this character connects to four other joints (tail, spine and two legs), and so it produces four bones.

be moved very efficiently with a skeleton. However, the motion of a face tends to be much more complex, with the motions of individual vertices being more independent. To convincingly animate a face using the skeletal approach, the required number of joints approaches the number of vertices in the mesh, thus diminishing its effectiveness as a compression technique. This is one reason why morph target techniques are often favored over the skeletal approach for facial animation. (Another common reason is that morph targets tend to be a more natural way for animators to work.)

12.2 Skeletons

A skeleton is comprised of a *hierarchy* of rigid pieces known as *joints*. In the game industry, we often use the terms “joint” and “bone” interchangeably, but the term *bone* is actually a misnomer. Technically speaking, the joints are the objects that are directly manipulated by the animator, while the bones are simply the empty spaces between the joints. As an example, consider the pelvis joint in the Crank the Weasel character model. It is a single joint, but because it connects to four other joints (the tail, the spine and the left and right hip joints), this one joint appears to have four bones sticking out of it. This is shown in more detail in Figure 12.5. Game engines don’t care a whit about bones—only the joints matter. So whenever you hear the term “bone” being used in the industry, remember that 99% of the time we are actually speaking about joints.

12.2.1 The Skeleal Hierarchy

As we've mentioned, the joints in a skeleton form a hierarchy or tree structure. One joint is selected as the root, and all other joints are its children, grandchildren and so on. A typical joint hierarchy for skinned animation looks almost identical to a typical rigid hierarchy. For example, a humanoid character's joint hierarchy might look something like the one depicted in Figure 12.6.

We usually assign each joint an index from 0 to $N - 1$. Because each joint has one and only one parent, the hierarchical structure of a skeleton can be fully described by storing the index of its parent with each joint. The root joint has no parent, so its parent index is usually set to an invalid value such as -1 .

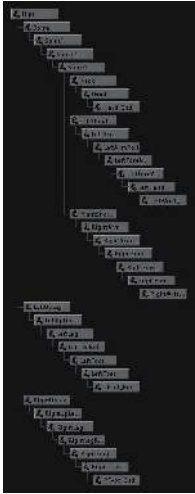


Figure 12.6. Example of a skeletal hierarchy, as it would appear in Maya's Hypergraph Hierarchy view.

12.2.2 Representing a Skeleton in Memory

A skeleton is usually represented by a small top-level data structure that contains an array of data structures for the individual joints. The joints are usually listed in an order that ensures a child joint will always appear after its parent in the array. This implies that joint zero is always the root of the skeleton.

Joint indices are usually used to refer to joints within animation data structures. For example, a child joint typically refers to its parent joint by specifying its index. Likewise, in a skinned triangle mesh, a vertex refers to the joint or joints to which it is bound by index. This is much more efficient than referring to joints by name, both in terms of the amount of storage required (a joint index can be 8 bits wide, as long as we are willing to accept a maximum of 256 joints per skeleton) and in terms of the amount of time it takes to look up a referenced joint (we can use the joint index to jump immediately to a desired joint in the array).

Each joint data structure typically contains the following information:

- The *name* of the joint, either as a string or a hashed 32-bit string id.
- The *index* of the joint's *parent* within the skeleton.
- The *inverse bind pose transform* of the joint. The bind pose of a joint is the position, orientation and scale of that joint at the time it was bound to the vertices of the skin mesh. We usually store the *inverse* of this transformation for reasons we'll explore in more depth in the following sections.

A typical skeleton data structure might look something like this:

```
struct Joint
{
    Matrix4x3    m_invBindPose; // inverse bind pose
```

```

                                // transform
    const char* m_name;          // human-readable joint
                                // name
    U8          m_iParent;       // parent index or 0xFF
                                // if root
};

struct Skeleton
{
    U32          m_jointCount;    // number of joints
    Joint*      m_aJoint;        // array of joints
};

```

12.3 Poses

No matter what technique is used to produce an animation, be it cel-based, rigid hierarchical or skinned/skeletal, every animation takes place over time. A character is imbued with the illusion of motion by arranging the character's body into a sequence of discrete, still *poses* and then displaying those poses in rapid succession, usually at a rate of 30 or 60 *poses per second*. (Actually, as we'll see in Section 12.4.1.1, we often *interpolate* between adjacent poses rather than displaying a single pose verbatim.) In skeletal animation, the pose of the skeleton directly controls the vertices of the mesh, and posing is the animator's primary tool for breathing life into her characters. So clearly, before we can animate a skeleton, we must first understand how to *pose* it.

A skeleton is posed by rotating, translating and possibly scaling its joints in arbitrary ways. The *pose* of a joint is defined as the joint's position, orientation and scale, relative to some frame of reference. A joint pose is usually represented by a 4×4 or 4×3 matrix, or by an SRT data structure (scale, quaternion rotation and vector translation). The pose of a skeleton is just the set of all of its joints' poses and is normally represented as a simple array of matrices or SRTs.

12.3.1 Bind Pose

Two different poses of the same skeleton are shown in Figure 12.7. The pose on the left is a special pose known as the *bind pose*, also sometimes called the *reference pose* or the *rest pose*. This is the pose of the 3D mesh prior to being bound to the skeleton (hence the name). In other words, it is the pose that the mesh would assume if it were rendered as a regular, unskinned triangle mesh, without any skeleton at all. The bind pose is also called the *T-pose* because

the character is usually standing with his feet slightly apart and his arms outstretched in the shape of the letter T. This particular stance is chosen because it keeps the limbs away from the body and each other, making the process of binding the vertices to the joints easier.

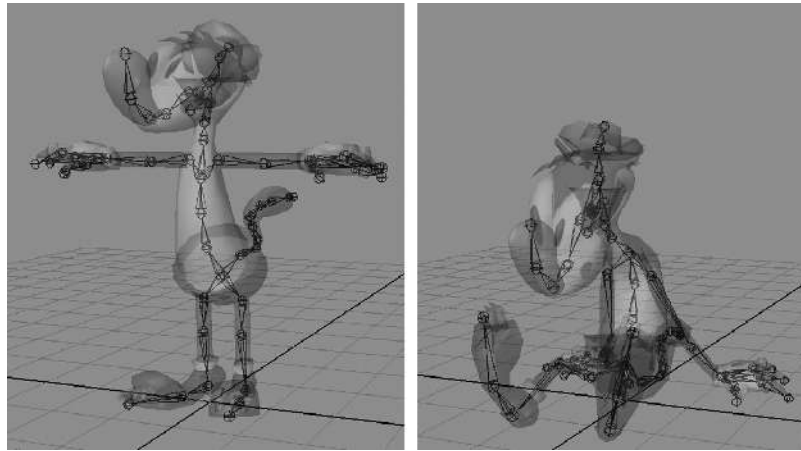


Figure 12.7. Two different poses of the same skeleton. The pose on the left is the special pose known as *bind pose*.

12.3.2 Local Poses

A joint's pose is most often specified relative to its *parent* joint. A parent-relative pose allows a joint to move naturally. For example, if we rotate the shoulder joint, but leave the parent-relative poses of the elbow, wrist and fingers unchanged, the entire arm will rotate about the shoulder in a rigid manner, as we'd expect. We sometimes use the term *local pose* to describe a parent-relative pose. Local poses are almost always stored in SRT format, for reasons we'll explore when we discuss animation blending.

Graphically, many 3D authoring packages like Maya represent joints as small spheres. However, a joint has a rotation and a scale, not just a translation, so this visualization can be a bit misleading. In fact, a joint actually defines a coordinate space no different in principle from the other spaces we've encountered (like model space, world space or view space). So it is best to picture a joint as a set of Cartesian coordinate axes. Maya gives the user the option of displaying a joint's local coordinate axes—this is shown in Figure 12.8.

Mathematically, a joint pose is nothing more than an *affine transformation*. The pose of joint j can be written as the 4×4 affine transformation matrix $\mathbf{P}_j$, which is comprised of a translation vector $\mathbf{T}_j$, a 3×3 diagonal scale matrix

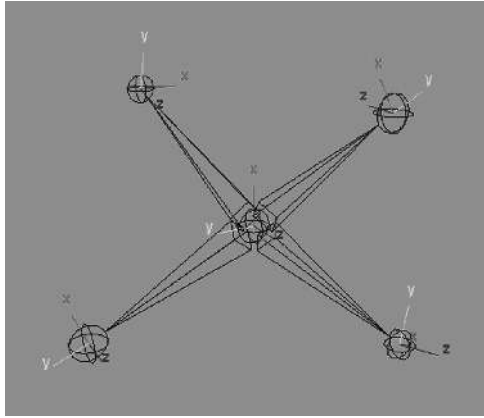


Figure 12.8. Every joint in a skeletal hierarchy defines a set of local coordinate space axes, known as joint space.

$\mathbf{S}_j$ and a 3×3 rotation matrix $\mathbf{R}_j$. The pose of an entire skeleton $\mathbf{P}^{\text{skel}}$ can be written as the set of all poses $\mathbf{P}_j$, where j ranges from 0 to $N - 1$:

$$\mathbf{P}_j = \begin{bmatrix} \mathbf{S}_j \mathbf{R}_j & 0 \\ \mathbf{T}_j & 1 \end{bmatrix},$$

$$\mathbf{P}^{\text{skel}} = \left\{ \mathbf{P}_j \right\}_{j=0}^{N-1}.$$

12.3.2.1 Joint Scale

Some game engines assume that joints will never be scaled, in which case $\mathbf{S}_j$ is simply omitted and assumed to be the identity matrix. Other engines make the assumption that scale will be *uniform* if present, meaning it is the same in all three dimensions. In this case, scale can be represented using a single scalar value s_j . Some engines even permit *nonuniform* scale, in which case scale can be compactly represented by the three-element vector $\mathbf{s}_j = [s_{jx} \ s_{jy} \ s_{jz}]$. The elements of the vector $\mathbf{s}_j$ correspond to the three diagonal elements of the 3×3 scaling matrix $\mathbf{S}_j$, so it is not really a vector *per se*. Game engines almost never permit shear, so $\mathbf{S}_j$ is almost never represented by a full 3×3 scale/shear matrix, although it certainly *could* be.

There are a number of benefits to omitting or constraining scale in a pose or animation. Clearly using a lower-dimensional scale representation can save memory. (Uniform scale requires a single floating-point scalar per joint per animation frame, while nonuniform scale requires three floats, and a full 3×3 scale-shear matrix requires nine.) Restricting our engine to uniform scale has the added benefit of ensuring that the bounding sphere of a joint will never

be transformed into an ellipsoid, as it could be when scaled in a nonuniform manner. This greatly simplifies the mathematics of frustum and collision tests in engines that perform such tests on a per-joint basis.

12.3.2.2 Representing a Joint Pose in Memory

As we mentioned above, joint poses are usually stored in SRT format. In C++, such a data structure might look like this, where *Q* is first to ensure proper alignment and optimal structure packing. (Can you see why?)

```
struct JointPose
{
    Quaternion m_rot;    // R
    Vector3    m_trans;  // T
    F32        m_scale;  // S (uniform scale only)
};
```

If nonuniform scale is permitted, we might define a joint pose like this instead:

```
struct JointPose
{
    Quaternion m_rot;    // R
    Vector4    m_trans;  // T
    Vector4    m_scale;  // S
};
```

The local pose of an entire skeleton can be represented as follows, where it is understood that the array *m_aLocalPose* is dynamically allocated to contain just enough occurrences of *JointPose* to match the number of joints in the skeleton.

```
struct SkeletonPose
{
    Skeleton* m_pSkeleton; // skeleton + num joints
    JointPose* m_aLocalPose; // local joint poses
};
```

12.3.2.3 The Joint Pose as a Change of Basis

It's important to remember that a *local* joint pose is specified relative to the joint's immediate parent. Any affine transformation can be thought of as transforming points and vectors from one coordinate space to another. So when the joint pose transform P_j is applied to a point or vector that is expressed in the coordinate system of the joint *j*, the result is that same point or vector expressed in the space of the parent joint.

As we've done in earlier chapters, we'll adopt the convention of using subscripts to denote the direction of a transformation. Since a joint pose takes points and vectors from the *child* joint's space (C) to that of its *parent* joint (P), we can write it $(\mathbf{P}_{C \rightarrow P})_j$. Alternatively, we can introduce the function $p(j)$, which returns the parent index of joint j , and write the local pose of joint j as $\mathbf{P}_{j \rightarrow p(j)}$.

On occasion we will need to transform points and vectors in the opposite direction—from *parent space* into the space of the *child* joint. This transformation is just the inverse of the local joint pose. Mathematically, $\mathbf{P}_{p(j) \rightarrow j} = (\mathbf{P}_{j \rightarrow p(j)})^{-1}$.

12.3.3 Global Poses

Sometimes it is convenient to express a joint's pose in model space or world space. This is called a *global pose*. Some engines express global poses in matrix form, while others use the SRT format.

Mathematically, the model-space pose of a joint ($j \rightarrow M$) can be found by walking the skeletal hierarchy from the joint in question all the way to the root, multiplying the local poses ($j \rightarrow p(j)$) as we go. Consider the hierarchy shown in Figure 12.9. The parent space of the root joint is defined to be model space, so $p(0) \equiv M$. The model-space pose of joint J_2 can therefore be written as follows:

$$\mathbf{P}_{2 \rightarrow M} = \mathbf{P}_{2 \rightarrow 1} \mathbf{P}_{1 \rightarrow 0} \mathbf{P}_{0 \rightarrow M}.$$

Likewise, the model-space pose of joint J_5 is just

$$\mathbf{P}_{5 \rightarrow M} = \mathbf{P}_{5 \rightarrow 4} \mathbf{P}_{4 \rightarrow 3} \mathbf{P}_{3 \rightarrow 0} \mathbf{P}_{0 \rightarrow M}.$$

In general, the global pose (joint-to-model transform) of any joint j can be written as follows:

$$\mathbf{P}_{j \rightarrow M} = \prod_{i=j}^0 \mathbf{P}_{i \rightarrow p(i)}, \quad (12.1)$$

where it is understood that i becomes $p(i)$ (the parent of joint i) after each iteration in the product, and $p(0) \equiv M$.

12.3.3.1 Representing a Global Pose in Memory

We can extend our `SkeletonPose` data structure to include the global pose as follows, where again we dynamically allocate the `m_aGlobalPose` array based on the number of joints in the skeleton: